HASH TABLE

SECOND-WEEK ASSIGNMENT FOR THE DATA STRUCTURE

Lecturer :

Andi Iwan Nurhidayat, S.Kom., M.T.

Presented by :

Aisyah Currents

25091397108 / 2025I

INFORMATIC MANAGEMENT MAJOR

FACULTY OF VOCATION

UNIVERSITAS NEGERI SURABAYA

YEAR 2026

The Source Code

```
"""
Hash Table Premium+ (Linear Probing) — matplotlib + keyboard controls
(Windows CMD safe)
Controls:
  SPACE : pause/resume
  RIGHT : step forward (when paused)
  LEFT  : step backward (when paused)
  R     : restart
  ESC/Q : quit
Run:
  python hash_table_premium_keyboard.py
Options:
  python hash_table_premium_keyboard.py --size 12 --nkeys 11
  python hash_table_premium_keyboard.py --save gif
"""
import argparse
import random
import numpy as np
import matplotlib
matplotlib.use("TkAgg")  # CMD Windows GUI backend
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
def plan_inserts(keys, size=20):
    table = [None] * size
    frame_data = []
    load_factors = []
    for step, key in enumerate(keys, 1):
        start = hash(key) % size
        idx = start
        probes = 0
        def snap(phase, placed=False):
            frame_data.append({
                "phase": phase,
                "step": step,
                "key": key,
                "start": start,
                "idx": idx,
                "probes": probes,
                "table": list(table),
                "placed": placed
            })
        snap("start", placed=False)
        while table[idx] is not None:
            snap("collision", placed=False)
            probes += 1
            idx = (idx + 1) % size
            snap("probe", placed=False)
        table[idx] = key
        snap("place", placed=True)
        lf = sum(v is not None for v in table) / size
        load_factors.append(lf)
        snap("pause", placed=True)
    return frame_data, load_factors
def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--size", type=int, default=20)
    parser.add_argument("--nkeys", type=int, default=18)
    parser.add_argument("--seed", type=int, default=42)
    parser.add_argument("--save", choices=["none", "gif"], default="none")
```

```
    args = parser.parse_args()
    random.seed(args.seed)
    keys = [f"k{i}" for i in range(args.nkeys)]
    random.shuffle(keys)
    frames, load_factors = plan_inserts(keys, size=args.size)
    # ==== FIGURE LAYOUT ====
    fig = plt.figure(figsize=(12, 6), constrained_layout=True)
    gs = fig.add_gridspec(2, 1, height_ratios=[3, 1])
    ax_table = fig.add_subplot(gs[0])
    ax_lf = fig.add_subplot(gs[1])
    fig.suptitle("Hash Table Premium+ (Linear Probing) — Keyboard Control",
fontsize=14)
    # ---- TABLE GRID ----
    n = args.size
    ax_table.set_xlim(-0.5, n - 0.5)
    ax_table.set_ylim(-0.5, 0.5)
    ax_table.set_yticks([])
    ax_table.set_xticks(range(n))
    ax_table.set_xlabel("Index Bucket")
    rects, texts = [], []
    for i in range(n):
        r = plt.Rectangle((i - 0.5, -0.35), 1.0, 0.7, fill=True, alpha=0.25)
        ax_table.add_patch(r)
        rects.append(r)
        t = ax_table.text(i, 0, "", ha="center", va="center", fontsize=9)
        texts.append(t)
    info  =  ax_table.text(0.01,  1.08,  "",  transform=ax_table.transAxes,
fontsize=11)
    legend = ax_table.text(
        0.01, 1.01,
        "SPACE: pause/resume | ←/→: step (pause) | R: restart | Q/Esc: quit",
        transform=ax_table.transAxes, fontsize=9
    )
    # ---- LOAD FACTOR CHART ----
    ax_lf.set_xlim(1, max(2, args.nkeys))
    ax_lf.set_ylim(0, 1.05)
    ax_lf.set_xlabel("Insert ke-")
    ax_lf.set_ylabel("Load Factor")
    ax_lf.grid(True, alpha=0.25)
    lf_line, = ax_lf.plot([], [])
    lf_dot, = ax_lf.plot([], [], marker="o", linestyle="")
    bar_bg      =      plt.Rectangle((0.02,      0.15),      0.96,      0.2,
transform=ax_lf.transAxes, alpha=0.15)
    bar_fg      =      plt.Rectangle((0.02,      0.15),      0.00,      0.2,
transform=ax_lf.transAxes, alpha=0.35)
    ax_lf.add_patch(bar_bg)
    ax_lf.add_patch(bar_fg)
    lf_text   =   ax_lf.text(0.02,   0.45,   "",   transform=ax_lf.transAxes,
fontsize=10)
    # ==== PLAYER STATE ====
    state = {
        "i": 0,          # current frame index
        "paused": False  # paused flag
    }
    def render(frame):
        """Render a single frame dict."""
        table = frame["table"]
        start = frame["start"]
        idx = frame["idx"]
        phase = frame["phase"]
        # reset buckets
```

```
        for j in range(n):
            rects[j].set_alpha(0.25)
            rects[j].set_linewidth(1.0)
            texts[j].set_text("" if table[j] is None else str(table[j]))
        # highlight target hash bucket
        rects[start].set_alpha(0.55)
        rects[start].set_linewidth(2.5)
        # highlight current idx
        rects[idx].set_alpha(0.75)
        rects[idx].set_linewidth(3.0)
        # collision emphasis
        if phase == "collision":
            rects[idx].set_alpha(0.9)
        filled = sum(v is not None for v in table)
        lf = filled / n
        info.set_text(
            f"Frame       {state['i']+1}/{len(frames)}       |       Step
{frame['step']}/{args.nkeys} | key='{frame['key']}' | "
            f"hash%size={start}  |  idx={idx}  |  probes={frame['probes']}  |
load_factor={lf:.2f} | phase={phase} | "
            f"{'PAUSED' if state['paused'] else 'PLAY'}"
        )
        # completed inserts so far
        completed = frame["step"] - 1 + (1 if frame["placed"] else 0)
        if completed <= 0:
            x, y = [], []
        else:
            x = list(range(1, completed + 1))
            y = load_factors[:completed]
        lf_line.set_data(x, y)
        if x:
            lf_dot.set_data([x[-1]], [y[-1]])
            bar_fg.set_width(0.96 * y[-1])
            lf_text.set_text(f"Load    Factor:    {y[-1]:.2f}    ({int(y[-
1]*100)}%)")
        else:
            lf_dot.set_data([], [])
            bar_fg.set_width(0.00)
            lf_text.set_text("Load Factor: 0.00 (0%)")
        return rects + texts + [info, legend, lf_line, lf_dot, bar_fg,
lf_text]
    def update(_):
        """Animation tick: advance if not paused; otherwise keep frame."""
        if not state["paused"]:
            state["i"] = min(state["i"] + 1, len(frames) - 1)
        return render(frames[state["i"]])
    # Init render
    def init():
        return render(frames[state["i"]])
    ani = FuncAnimation(
        fig, update, init_func=init,
        frames=np.arange(len(frames)),     #  timer  ticks,  actual  frame
controlled by state["i"]
        interval=450, blit=False, repeat=False
    )
    def on_key(event):
        k = event.key
        if k == " ":
            state["paused"] = not state["paused"]
            fig.canvas.draw_idle()
        elif k == "right":
```

```
            if state["paused"]:
                state["i"] = min(state["i"] + 1, len(frames) - 1)
                render(frames[state["i"]])
                fig.canvas.draw_idle()
        elif k == "left":
            if state["paused"]:
                state["i"] = max(state["i"] - 1, 0)
                render(frames[state["i"]])
                fig.canvas.draw_idle()
        elif k in ("r", "R"):
            state["i"] = 0
            state["paused"] = True  # restart dalam kondisi pause biar enak
step
            render(frames[state["i"]])
            fig.canvas.draw_idle()
        elif k in ("escape", "q", "Q"):
            plt.close(fig)
    fig.canvas.mpl_connect("key_press_event", on_key)
    if args.save == "gif":
        try:
            from matplotlib.animation import PillowWriter
            ani.save("hash_table_premium_keyboard.gif",
writer=PillowWriter(fps=2))
            print("Tersimpan: hash_table_premium_keyboard.gif")
        except Exception as e:
            print("Gagal simpan GIF. Pastikan 'pillow' terpasang. Error:",
e)
    plt.show()
if __name__ == "__main__":
    main()
```

Please provide an explanation of the code above using a simple example (like a car park).

The code is a simulation and visualization of how a hash table works using the linear probing collision resolution technique. It uses the matplotlib library to create an animated graphical display and allows the user to control the animation with keyboard input. The main purpose of the program is to help users clearly see how keys are inserted into a hash table, how collisions occur, and how the table behaves as it becomes more filled.

To understand the idea more easily, imagine a parking lot with a fixed number of parking spaces arranged in a row. Each parking space has a number. In this analogy, the parking lot represents the hash table, and each parking space represents a bucket or slot inside the table. The cars represent the keys that will be stored. When a car arrives, a guard assigns it a suggested parking space based on a rule. In the program, this rule is calculated using the expression hash(key) modulo size. This determines the starting index where the key should be placed.

If the suggested parking space is empty, the car parks there immediately. However, if another car is already parked in that space, a collision occurs. In that situation, the car does not leave the parking lot. Instead, it moves to the next space on the right to check whether it is

empty. If that space is also occupied, it continues moving to the next one, repeating the process until it finds an empty space. This step by step movement to the next index is called linear probing. If the car reaches the last parking space and still has not found an empty one, it wraps around to the first space because of the modulo operation used in the calculation.

The function plan_inserts is responsible for simulating this entire process before it is shown on the screen. It goes through each key one by one and records every important moment. These moments include when a key first tries its hashed index, when a collision is detected, when probing moves to the next index, and when the key is finally placed into the table. Each of these moments is stored as a frame. Later, these frames are used by the animation system to visually replay the entire insertion process step by step.

In the visual part of the program, each slot of the hash table is drawn as a rectangle. When a key is being inserted, the originally hashed index is highlighted so the user can see where the key first attempted to go. The current index being checked during probing is also highlighted more strongly. If a collision happens, the visual emphasis changes to show that the slot is already occupied. This makes it easier to understand how the algorithm searches for an available position.

The program also calculates and displays something called the load factor. The load factor is the number of filled slots divided by the total number of slots. Returning to the parking lot analogy, the load factor shows how full the parking lot is. If the parking lot has ten spaces and five are occupied, the load factor is zero point five. As the load factor increases, collisions become more common because fewer empty spaces are available. This means that keys may need to probe more positions before finding a free slot. The program shows this visually in a graph below the table, allowing users to see how the load factor grows after each insertion.

The animation is controlled by a state system that keeps track of the current frame and whether the simulation is paused or running. The update function advances the frame automatically if the animation is not paused. Keyboard controls make the simulation interactive. The spacebar pauses or resumes the animation. The right arrow key moves forward one step when paused. The left arrow key moves backward one step. The R key restarts the simulation from the beginning, and the escape or Q key closes the window. This interactivity allows users to carefully observe each stage of the insertion process.

Overall, this code is an educational tool that demonstrates fundamental concepts in data structures. It shows how a hash function determines an index, how collisions happen when two keys map to the same index, how linear probing resolves those collisions, and how performance is affected as the load factor increases. By visualizing each step, the program helps students understand not only what a hash table does, but also why its efficiency depends on maintaining enough empty space.